

Trabalho 03: Sistema de Matchmaking

Estrutura de Dados

Rafael Gontijo Ferreira

rafael.ferreira.2@fgv.edu.br

Rudá Dantas Ruoso Brandão

rudabrandao@ruoso.com

1 Descrição do Projeto

Este trabalho apresenta a implementação de um sistema de *matchmaking* para jogadores online, desenvolvido como terceira atividade de Estrutura de Dados. O objetivo central é gerenciar uma fila de espera de jogadores e formar grupos com níveis de habilidade próximos, explorando algoritmos de ordenação como ferramenta central para tornar essa busca eficiente.

O sistema suporta as seguintes operações fundamentais:

- **Inserção:** Adiciona um jogador ao final da fila de espera.
- **Remoção:** Remove um jogador da fila de espera pelo seu identificador único.
- **Recuperação:** Recuperação de uma cópia da lista de espera atual.
- **Exibição:** Exibição da fila de espera atual.
- **Ordenação:** Ordena a fila por score crescente; desempate por timestamp.
- **Formação de grupo:** Retorna o primeiro grupo válido de jogadores consecutivos na fila ordenada.

2 Organização do Sistema

O projeto é modularizado nos seguintes arquivos:

- `main.cpp`: Ponto de entrada com testes funcionais e benchmark de desempenho.
- `Player (.hpp/.cpp)`: Declaração e implementação da classe `Player`
- `Matchmaking (.hpp/.cpp)`: Declaração e implementação da classe `Matchmaking`.
- `grafico.ipynb`: Notebook Python para geração dos gráficos de desempenho com os dados de benchmark.

O projeto compila com:

```
g++ main.cpp Matchmaking.cpp Player.cpp -o matchmaking
```

3 Estruturas de Dados e Implementação

3.1 Array Dinâmico

A classe `Matchmaking` armazena os jogadores em um array alocado dinamicamente no construtor com capacidade `MAX_PLAYERS`, e liberado no destrutor:

```

players = new Player [MAX_PLAYERS];    // construtor
delete [] players;                      // destrutor

```

Essa abordagem evita o risco de estouro de pilha que ocorreria com um array estático de 100.000 elementos do tipo `Player` declarado diretamente no corpo da classe.

3.2 Inserção e Remoção

A **inserção** coloca o novo jogador ao final do array e incrementa o contador de tamanho, sendo uma operação $O(1)$. A **remoção** por ID percorre o array linearmente e, ao encontrar o elemento, desloca todos os subsequentes uma posição para a esquerda, compactando o array, tendo custo $O(n)$. Essa compactação é necessária para não corromper a lista e permitir o funcionamento da lógica de remoção de grupos.

4 Algoritmos de Ordenação

Ambos os algoritmos ordenam os jogadores por score crescente e, em caso de empate, por timestamp crescente. Nenhuma função de ordenação da STL foi utilizada.

4.1 Insertion Sort

O *Insertion Sort* mantém um prefixo ordenado do array. A cada iteração i , o elemento na posição i é inserido na posição correta dentro do prefixo $[0, i - 1]$ por meio de trocas sucessivas com seu vizinho esquerdo, interrompidas quando a condição de ordem é satisfeita. Em caso de empate de score, é avaliado o timestamp de cada jogador.

Custo: $O(n^2)$ no pior caso e, para arrays quase ordenados, $O(n \cdot k)$, onde k representa a distância máxima entre um jogador e sua posição correta.

4.2 Merge Sort

O *Merge Sort* é implementado recursivamente por uma função auxiliar `sortByScoreMerge(low_bound, up_bound)`. A cada chamada, o segmento é dividido ao meio, cada metade é ordenada recursivamente, e os dois segmentos são intercalados em um array auxiliar alocado dinamicamente, copiado de volta ao array principal e então liberado. Novamente, a intercalação aplica os dois critérios de comparação: score e, em empate, timestamp.

Custo: $O(n \log n)$ em todos os casos. A cada nível da árvore de recursão é realizada a intercalação sobre o array completo (dividido em vários segmentos), o que tem custo $O(n)$. O processo de divisão gera $O(\log n)$ níveis.

5 Formação de Grupos

O método `formGroup(groupSize, delta, n)` assume uma ordenação prévia dos jogadores. Dado que o array está ordenado por score, em qualquer janela contígua de `groupSize` elementos o menor score está na primeira posição e o maior na última. Portanto, basta verificar a diferença de score entre o primeiro e último jogador da janela:

```

players[i + groupSize - 1].getScore() - players[i].getScore() <= delta

```

O método percorre o array com uma **janela deslizante** de tamanho `groupSize`, testando cada posição inicial i . Ao encontrar a primeira janela válida, copia os jogadores para um novo array alocado dinamicamente e os remove da fila via `removePlayer`. A liberação do ponteiro retornado fica a cargo de quem chama a função.

Sem a ordenação prévia, seria necessário testar todas as combinações de g elementos, tendo um custo $O(n^g)$. Com a ordenação, a busca pela janela é $O(n)$ e as remoções custam $O(g \cdot n)$, resultando em custo total $O(n \log n + g \cdot n)$ para uma ordenação via *Merge Sort*.

6 Custo Computacional

Operação	Complexidade	Observação
Inserção	$O(1)$	Inserção ao final do array.
Remoção por ID	$O(n)$	Busca linear + compactação.
Insertion Sort	$O(n^2)$ / $O(n)$	Pior / melhor caso.
Merge Sort	$O(n \log n)$	Todos os casos.
Formação de grupo	$O(n + g \cdot n)$	Busca $O(n)$ + remoções $O(g \cdot n)$.
<code>getWaitingPlayers</code>	$O(n)$	Cópia integral do array interno.

Tabela 1: Custo teórico para as funções implementadas

7 Análise Experimental: Insertion Sort vs. Merge Sort

7.1 Metodologia

Foram realizadas 10 amostras de teste, qm que, para cada uma foram geradas aleatórias por meio de `std::mt19937` com `seed` variável (`std::random_device`), com tamanho inicial `MAX_PLAYERS` (100.000) reduzido geometricamente por um fator $2/3$ a cada iteração até $n = 1$. Dentro de cada iteração de teste, as funções de *Insertion* e *Merge Sort* foram avaliadas com a mesma entrada aleatória. Os resultados foram armazenados em `out.csv` e analisados em `python` via `seaborn` e `pandas`

7.2 Resultados

n (jogadores)	Insertion Sort (s)	Merge Sort (s)	Razão
100.000	$\approx 104,7$	$\approx 0,15$	$\sim 698\times$
66.666	$\approx 47,3$	$\approx 0,095$	$\sim 497\times$
44.444	$\approx 20,0$	$\approx 0,060$	$\sim 333\times$
29.629	$\approx 9,0$	$\approx 0,038$	$\sim 236\times$
19.752	$\approx 4,0$	$\approx 0,025$	$\sim 160\times$
13.168	$\approx 1,7$	$\approx 0,016$	$\sim 106\times$
8.778	$\approx 0,77$	$\approx 0,010$	$\sim 77\times$
5.852	$\approx 0,34$	$\approx 0,0065$	$\sim 52\times$
3.901	$\approx 0,15$	$\approx 0,0041$	$\sim 36\times$
2.600	$\approx 0,067$	$\approx 0,0027$	$\sim 24\times$
1.733	$\approx 0,029$	$\approx 0,0018$	$\sim 16\times$
1.155	$\approx 0,011$	$\approx 0,0011$	$\sim 10\times$
770	$\approx 0,0054$	$\approx 0,00072$	$\sim 7\times$
513	$\approx 0,0025$	$\approx 0,00045$	$\sim 5\times$

Tabela 2: Tempos médios observados (10 amostras, entradas aleatórias).

7.3 Discussão

Os resultados são consistentes com as complexidades teóricas. Para $n = 100.000$, o *Insertion Sort* levou cerca de 104,7 s, enquanto o *Merge Sort* completou o mesmo trabalho em aproximadamente 0,15 s, uma diferença de quase 700 vezes. Essa razão cresce com n , o que é esperado: a diferença entre $O(n^2)$ e $O(n \log n)$ é amplificada à medida que n aumenta.

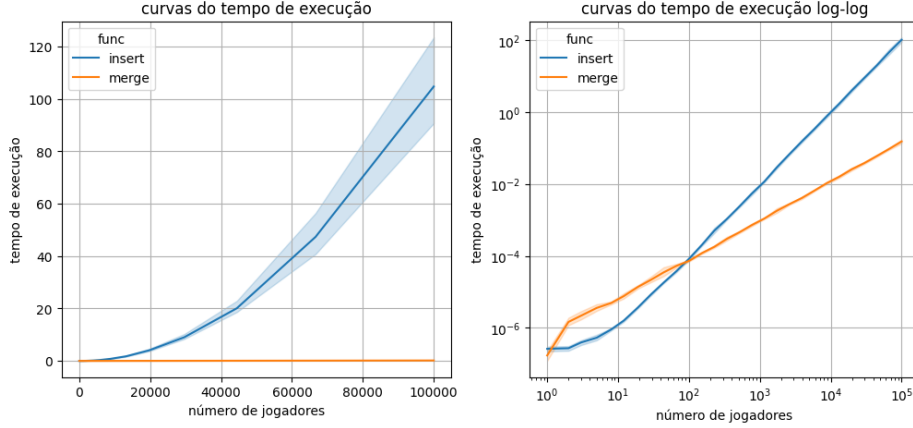


Figura 1: Curvas de tempo de execução

Nos gráficos em escala log-log, as curvas do Insertion Sort e do Merge Sort apresentam inclinações aproximadas de 2 e 1 (Figura 1), respectivamente, confirmando visualmente as complexidades teóricas. Para entradas pequenas ($n < 100$), a diferença absoluta de tempo é negligenciável apesar do *overhead* de recursão e alocação auxiliar do Merge Sort ser desfavorável.

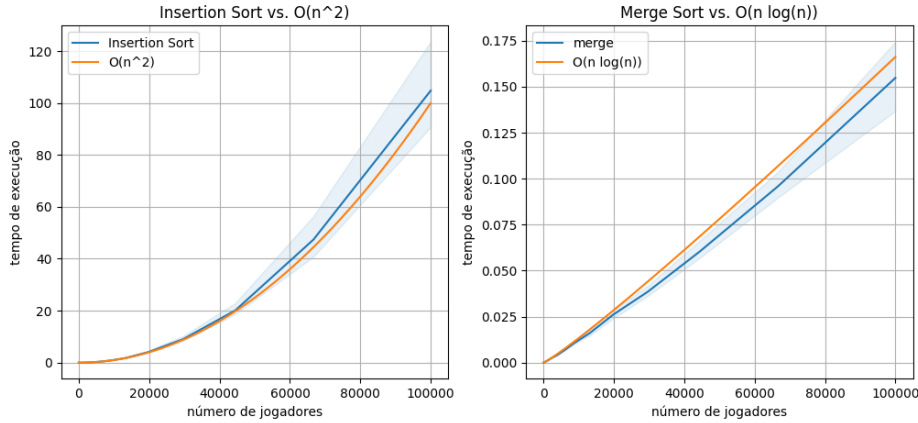


Figura 2: Custo medido x teórico

Além disso, comparando o desempenho das funções com a curva de suas complexidades teóricas, podemos reafirmar esse dado (Figura 2).

Em síntese, o Merge Sort é amplamente superior para filas de espera com muitos jogadores. O Insertion Sort tem relevância apenas como referência de implementação simples ou para listas já quase ordenadas, onde seu custo se aproxima de $O(n)$.